

교과목 4 : AI 활용 애플리케이션 개발

단원 2 : Django Framework

DAY2. Django 웹 애플리케이션 핵심 기능 구현

목 차

1. Django 전체 구조		3
2. Django App 만들기		9
3. Django Template Language		13
4. Model 과 ORM		45
5. Form 처리		50
6. Django Admin 과 권한 및 Session		54
7. 파일 관계 및 보안		59

1. Django 전체 구조

Django = Python 으로 웹사이트의 화면, 기능, 데이터베이스, 로그인 등을 체계적으로 연결해 주는 도구

1. Django 의 전체 구조

Django 는 기본적으로 MVT 패턴을 사용합니다.

MVT 는 다음 세 가지 요소로 구성됩니다.

구성 요소	의미	역할
Model	데이터	데이터베이스의 테이블과 데이터를 관리
View	기능 처리	사용자의 요청을 처리하고 결과를 반환
Template	화면	HTML 을 이용하여 브라우저에 화면 출력

Spring MVC 와 비교하면 대략 다음과 같습니다.

Spring MVC	Django
Model	Model
Controller	View
View	Template
DispatcherServlet	URL Dispatcher
JPA	Django ORM
Thymeleaf/JSP	Django Template

따라서 Django 의 View 는 일반적인 MVC 에서 말하는 Controller 역할에 가깝습니다.

웹 브라우저에서 요청하면 대략 다음 순서로 처리됩니다.



```
urls.py
  ↓
views.py
  ↓
models.py
  ↓
Database
  ↓
views.py
  ↓
Template HTML
  ↓
브라우저
```

예를 들어 사용자가 다음 주소에 접속한다고 가정하겠습니다.

```
http://127.0.0.1:8000/board/
```

Django 는 먼저 URL 을 확인합니다.

```
path("board/", views.board_list)
```

그 다음 views.py 의 함수를 실행합니다.

```
def board_list(request):
    ...
```

필요하면 Model 을 통해 데이터베이스에서 게시글을 가져옵니다.

```
boards = Board.objects.all()
```

그리고 HTML 로 데이터를 전달합니다.

```
return render(
    request,
    "board/list.html",
    {"boards": boards}
)
```

최종적으로 사용자 브라우저에 게시글 목록이 표시됩니다.

2. Django 의 Project 와 App

Django 를 처음 접할 때 가장 혼동하기 쉬운 개념이 Project 와 App 입니다.

Project

Project 는 전체 웹사이트입니다.

예를 들어 쇼핑몰을 만든다면 프로젝트는 다음과 같이 생각할 수 있습니다.

```
shoppingmall
```

프로젝트 안에는 전체 환경설정과 URL 설정 등이 들어갑니다.

대표적인 파일은 다음과 같습니다.

```
shoppingmall/  
    settings.py  
    urls.py  
    asgi.py  
    wsgi.py
```

App

App 은 웹사이트 안의 개별 기능입니다.

쇼핑몰 프로젝트라면 다음과 같은 App 을 만들 수 있습니다.

```
shoppingmall  
├── member  
├── board  
├── product  
├── order  
└── payment
```

각 App 은 독립적인 기능을 담당합니다.

예를 들어:

member

- 회원가입
- 로그인
- 회원정보 수정

board

- 게시글 등록
- 게시글 조회
- 게시글 수정
- 게시글 삭제

product

- 상품 등록
- 상품 조회

즉,

Project

↓

여러 개의 App

구조입니다.

3. Django 프로젝트 생성

Django 프로젝트는 다음 명령으로 생성합니다.

```
django-admin startproject config .
```

프로젝트가 만들어지면 대략 다음 구조가 생성됩니다.

project/

- ├─ manage.py
- ├─ config/
 - ├─ __init__.py
 - ├─ settings.py

```
├─ urls.py
├─ asgi.py
└─ wsgi.py
```

각 파일의 역할을 알아두는 것이 중요합니다.

manage.py

Django 프로젝트를 관리하는 명령어 실행 파일입니다.

예:

```
python manage.py runserver
python manage.py migrate
python manage.py createsuperuser
python manage.py startapp board
```

Django 를 사용할 때 매우 자주 사용합니다.

settings.py

Django 프로젝트의 전체 설정 파일입니다.

예를 들어 다음 내용을 설정합니다.

```
설치된 App
데이터베이스
언어
시간대
정적 파일
미디어 파일
보안 설정
템플릿
```

예:

```
INSTALLED_APPS = [
    ...
    "board",
```

```
"member",  
]
```

urls.py

웹 주소를 관리합니다.

예:

```
urlpatterns = [  
    path("admin/", admin.site.urls),  
    path("board/", include("board.urls")),  
]
```


2. Django App 만들기

1. Django App 만들기

App 은 다음과 같이 생성합니다.

```
python manage.py startapp board
```

그러면 다음과 같은 구조가 생성됩니다.

```
board/  
|  
├─ migrations/  
|  
├─ admin.py  
├─ apps.py  
├─ models.py  
├─ tests.py  
└─ views.py
```

보통 여기에 직접 urls.py 를 추가합니다.

```
board/  
├─ migrations/  
|  
├─ admin.py  
├─ apps.py  
├─ models.py  
├─ urls.py  
└─ views.py
```

2. URL Routing

Django 에서는 사용자가 입력한 주소에 따라 어떤 기능을 실행할지 결정합니다.

이것을 **URL Routing** 이라고 합니다.

예를 들어:

```
/board/  
/board/10/  
/member/login/  
/member/join/
```

같은 주소를 각각 다른 기능과 연결할 수 있습니다.

```
# board/urls.py  
from django.urls import path  
from . import views  
  
urlpatterns = [  
    path("", views.board_list),  
    path("<int:pk>/", views.board_detail),  
]
```

첫 번째 URL:

```
/board/
```

접속 시

```
views.board_list
```

함수가 실행됩니다.

두 번째:

```
/board/10/
```

접속 시

```
views.board_detail
```

함수가 실행되고 10 이 pk 값으로 전달됩니다.

예:

```
def board_detail(request, pk):  
    print(pk)
```

3. View

Django 에서 View 는 사용자의 요청을 실제로 처리하는 부분입니다.

간단한 예:

```
from django.http import HttpResponse  
  
def hello(request):  
    return HttpResponse("Hello Django")
```

브라우저에서 호출하면:

```
Hello Django
```

가 출력됩니다.

하지만 실제 프로젝트에서는 대부분 HTML Template 을 사용합니다.

```
from django.shortcuts import render  
  
def home(request):  
    return render(request, "home.html")
```

이 코드는 다음 HTML 파일을 찾아서 실행합니다.

```
templates/home.html
```

4. Template

Template 은 사용자가 브라우저에서 보는 **HTML 화면**입니다.

예:

```
<!DOCTYPE html>  
<html>  
<head>
```

```
<title>Django</title>
</head>
<body>
<h1>Django 게시판</h1>
</body>
</html>
```

Django Template 에서는 Python 에서 전달받은 값을 사용할 수도 있습니다.

View:

```
def home(request):

    context = {
        "username": "홍길동"
    }

    return render(
        request,
        "home.html",
        context
    )
```

Template:

```
<h1>{{ username }}님 환영합니다.</h1>
```

브라우저 결과:

```
홍길동님 환영합니다.
```

여기에서

```
{{ username }}
```

은 Django Template 변수입니다.

3. Django Template Language

Django Template Language(DTL)은 Django에서 Python으로 처리한 데이터를 HTML 화면에 동적으로 출력하기 위해 사용하는 템플릿 문법입니다.

Django의 MVT(Model-View-Template) 구조에서 **Template** 영역을 담당하는 핵심 기술이며, HTML 문서 안에서 변수 출력, 조건문, 반복문, URL 생성, 템플릿 상속 등의 기능을 사용할 수 있습니다.

1. Django Template Language란?

일반 HTML은 미리 작성한 정적인 내용을 브라우저에 표시합니다.

```
<h1>홍길동님 환영합니다.</h1>
```

이렇게 작성하면 항상 **홍길동**이라는 동일한 값만 표시됩니다.

하지만 로그인한 사용자에 따라 이름을 다르게 표시해야 한다면 HTML에 데이터를 전달해야 합니다.

Django에서는 View에서 데이터를 Template으로 전달합니다.

```
def home(request):
    context = {
        "name": "홍길동"
    }

    return render(request, "home.html", context)
```

Template에서는 다음과 같이 사용합니다.

```
<h1>{{ name }}님 환영합니다.</h1>
```

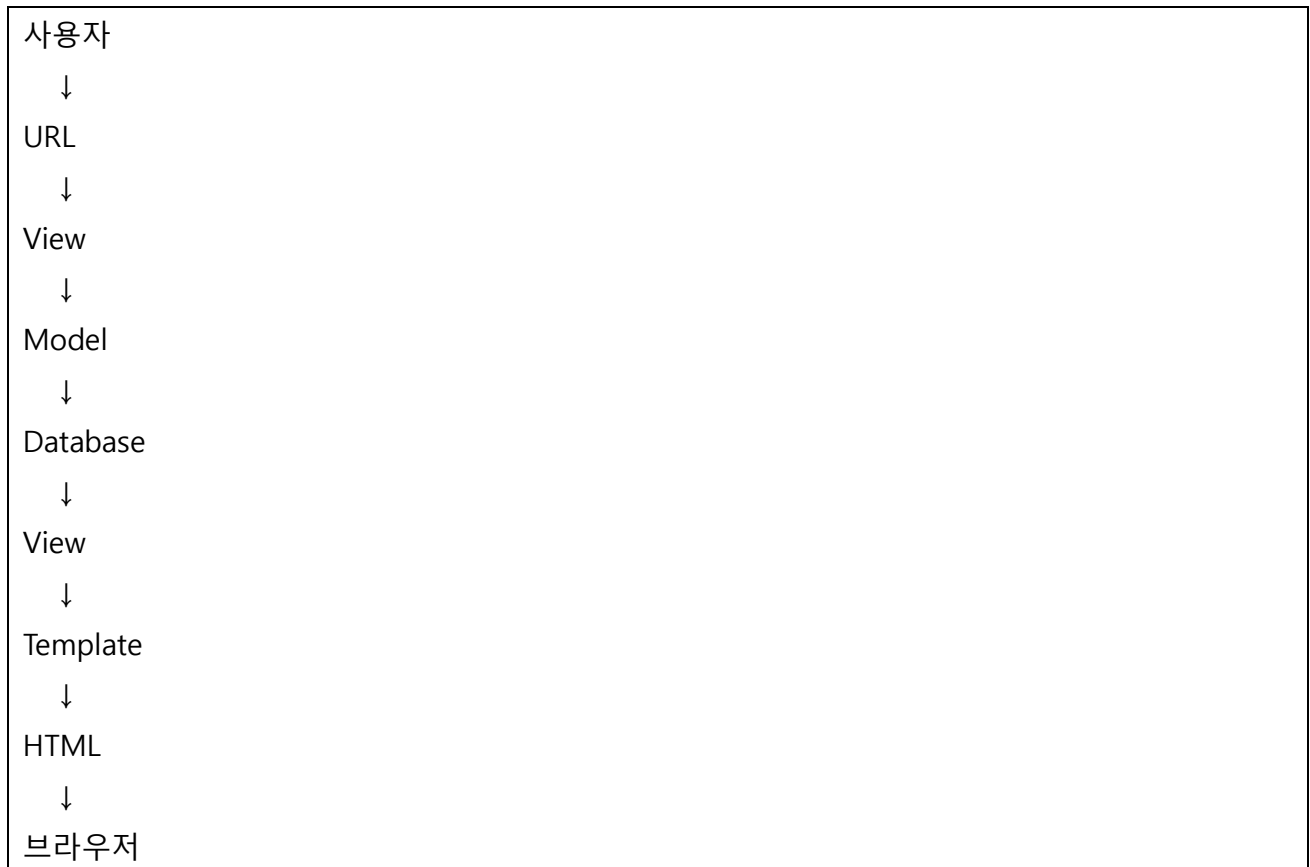
브라우저에는 다음과 같이 출력됩니다.

```
홍길동님 환영합니다.
```

즉, DTL은 View에서 전달받은 데이터를 HTML에 표현하기 위한 Django 전용 템플릿 문법입니다.

2. Django MVT와 Template의 관계

Django는 일반적으로 다음과 같은 **MVT 구조**를 사용합니다.



각 구성 요소의 역할은 다음과 같습니다.

구성 요소	역할
Model	데이터베이스와 데이터 처리
View	요청 처리 및 비즈니스 로직
Template	사용자에게 보여줄 HTML 화면
URL	요청 주소와 View 연결

Template에서는 가능하면 복잡한 프로그램 로직을 작성하지 않습니다.

예를 들어 데이터베이스에서 회원을 검색하는 작업은 View 또는 Model에서 처리하고, Template에서는 결과를 **출력하는 역할**에 집중합니다.

3. DTL의 기본 문법

DTL에서 가장 중요한 문법은 크게 다음 네 가지입니다.

문법	형태	용도
변수	{{ }}	데이터 출력
태그	{% %}	조건문, 반복문 등
필터		데이터 가공
주석	{# #}	주석 작성

대표적으로 다음과 같이 사용합니다.

```
{{ user.username }}

{% if user.is_authenticated %}
    로그인 상태
{% endif %}

{{ name|upper }}

{# Template 주석입니다. #}
```

이 네 가지 문법을 이해하면 DTL의 기본적인 사용법 대부분을 이해할 수 있습니다.

4. 변수 출력 {{ }}

가장 기본적인 기능은 **View에서 전달한 값을 HTML에 출력하는 것**입니다.

View

```
def home(request):

    context = {
        "name": "홍길동",
        "age": 25
    }

    return render(request, "home.html", context)
```

Template

```
<h1>{{ name }}님 환영합니다.</h1>

<p>나이 : {{ age }}</p>
```

결과는 다음과 같습니다.

```
홍길동님 환영합니다.

나이 : 25
```

기본 형식은 다음과 같습니다.

```
{{ 변수명 }}
```

5. 객체의 속성 접근

Template에서는 객체의 속성을 .으로 접근할 수 있습니다.

예를 들어 Django 사용자 객체가 있다면 다음과 같이 사용할 수 있습니다.

```
{{ user.username }}
{{ user.email }}
{{ user.first_name }}
```

로그인 사용자라면 다음과 같이 사용할 수 있습니다.

```
<h2>{{ user.username }}님 환영합니다.</h2>
```

Django의 request.user 객체는 인증 시스템과 연결되어 있기 때문에 로그인 화면에서 매우 자주 사용됩니다.

6. Dictionary 데이터 출력

View에서 Dictionary를 전달할 수도 있습니다.

```
def home(request):
    member = {
        "name": "홍길동",
```



```
        "age": 25,
        "email": "hong@test.com"
    }

    return render(
        request,
        "home.html",
        {"member": member}
    )
```

Template에서는 다음과 같이 접근합니다.

```
<p>이름 : {{ member.name }}</p>
<p>나이 : {{ member.age }}</p>
<p>이메일 : {{ member.email }}</p>
```

Python에서는 일반적으로 다음과 같이 작성합니다.

```
member["name"]
```

하지만 DTL에서는 다음처럼 작성할 수 있습니다.

```
{{ member.name }}
```

7. Template Tag {% %}

{% %}는 **Template Tag**라고 합니다.

변수를 단순히 출력하는 것이 아니라 프로그램의 흐름을 제어할 때 사용합니다.

대표적인 Template Tag에는 다음이 있습니다.

```
{% if %}
{% for %}
{% url %}
{% include %}
{% extends %}
{% block %}
{% csrf_token %}
{% load %}
```

예를 들어:

```
{% if user.is_authenticated %}  
    <p>로그인 사용자입니다.</p>  
{% endif %}
```

8. 조건문 {% if %}

DTL에서도 조건문을 사용할 수 있습니다.

기본 구조는 다음과 같습니다.

```
{% if 조건 %}  
    실행 내용  
{% endif %}
```

예를 들어:

```
{% if user.is_authenticated %}  
    <h2>{{ user.username }}님 환영합니다.</h2>  
{% endif %}
```

로그인한 사용자일 경우에만 이름을 출력합니다.

9. if / else

다음과 같이 else를 사용할 수 있습니다.

```
{% if user.is_authenticated %}  
    <p>{{ user.username }}님 로그인하셨습니다.</p>  
{% else %}  
    <p>로그인이 필요합니다.</p>  
{% endif %}
```

10. if / elif / else

여러 조건도 사용할 수 있습니다.

```
{% if score >= 90 %}  
    A등급  
{% elif score >= 80 %}  
    B등급  
{% elif score >= 70 %}  
    C등급  
{% else %}  
    재평가  
{% endif %}
```

Python과 비슷하지만 마지막에 반드시 다음을 작성해야 합니다.

```
{% endif %}
```

11. DTL에서 사용하는 비교 연산자

조건문에서는 다음과 같은 연산자를 사용할 수 있습니다.

연산자	의미
==	같다
!=	다르다
>	크다
<	작다
>=	크거나 같다
<=	작거나 같다
and	그리고
or	또는
not	부정
in	포함 여부

예를 들어:

```
{% if age >= 20 %}  
    성인입니다.  
{% endif %}
```

또는:

```
{% if user.is_authenticated and user.is_staff %}  
    관리자입니다.  
{% endif %}
```

12. 반복문 {% for %}

목록 데이터를 출력할 때 가장 많이 사용하는 문법입니다.

View에서 게시글 목록을 전달한다고 가정하겠습니다.

```
def board_list(request):  
    boards = [  
        {"title": "첫 번째 게시글"},  
        {"title": "두 번째 게시글"},  
        {"title": "세 번째 게시글"},  
    ]  
  
    return render(  
        request,  
        "board/list.html",  
        {"boards": boards}  
    )
```

Template:

```
{% for board in boards %}  
    <p>{{ board.title }}</p>  
{% endfor %}
```

결과:

```
첫 번째 게시물  
두 번째 게시물  
세 번째 게시물
```

기본 구조는 다음과 같습니다.

```
{% for 변수 in 목록 %}  
  
{% endfor %}
```

13. QuerySet과 반복문

실제 Django에서는 Model의 QuerySet을 반복문으로 출력하는 경우가 많습니다.

예를 들어:

```
boards = Board.objects.all()  
Template으로 전달합니다.  
return render(  
    request,  
    "board/list.html",  
    {"boards": boards}  
)
```

Template:

```
<table>  
    {% for board in boards %}  
        <tr>  
            <td>{{ board.id }}</td>  
            <td>{{ board.title }}</td>  
            <td>{{ board.writer }}</td>  
        </tr>  
    {% endfor %}  
</table>
```

이 방식은 Django CRUD 프로젝트에서 매우 중요합니다.

14. forloop 객체

반복문에서는 Django가 자동으로 forloop이라는 객체를 제공합니다.

```
{% for board in boards %}
    {{ forloop.counter }}
    {{ board.title }}
{% endfor %}
```

주요 속성은 다음과 같습니다.

속성	설명
forloop.counter	1부터 시작
forloop.counter0	0부터 시작
forloop.first	첫 번째 반복 여부
forloop.last	마지막 반복 여부
forloop.revcounter	역순 번호

예를 들어:

```
{% for board in boards %}
    <p>
        {{ forloop.counter }}.
        {{ board.title }}
    </p>
{% endfor %}
```

결과:

1. Django 공부
2. Python 공부
3. HTML 공부

15. {% empty %}

QuerySet이나 리스트가 비어 있을 경우를 처리할 수 있습니다.

```
{% for board in boards %}
    <p>{{ board.title }}</p>
{% empty %}
    <p>등록된 게시글이 없습니다.</p>
{% endfor %}
```

게시판 목록 화면에서 매우 유용합니다.

16. Template Filter

Filter는 출력할 데이터를 간단하게 가공할 때 사용합니다.

기본 구조는 다음과 같습니다.

```
{{ 변수|필터 }}
```

예:

```
{{ name|upper }}
```

name이 다음 값이라면:

```
django
```

결과는:

```
DJANGO
```

입니다.

17. 자주 사용하는 Filter

default

값이 없을 경우 기본값을 출력합니다.

```
{{ name|default:"이름 없음" }}
```

length

데이터의 개수를 출력합니다.

```
{{ boards|length }}
```

예:

```
<p>전체 게시글 : {{ boards|length }}개</p>
```

upper

대문자로 변경합니다.

```
{{ name|upper }}
```

lower

소문자로 변경합니다.

```
{{ name|lower }}
```

truncatechars

문자열을 지정된 길이로 줄입니다.

```
{{ board.content|truncatechars:20 }}
```

게시판 목록에서 긴 내용을 줄여서 표시할 때 유용합니다.

18. 날짜 출력 Filter

Django 프로젝트에서는 날짜 데이터를 자주 출력합니다.

```
{{ board.created_at }}
```

원하는 형식으로 출력하려면 date 필터를 사용할 수 있습니다.

```
{{ board.created_at|date:"Y-m-d" }}
```

결과:

```
2026-08-11
```


시간까지 출력하려면:

```
{{ board.created_at|date:"Y-m-d H:i" }}
```

결과:

```
2026-08-11 05:30
```

19. 여러 Filter 연결

Filter는 여러 개를 연결할 수도 있습니다.

```
{{ name|default:"guest"|upper }}
```

처리 순서는 왼쪽에서 오른쪽입니다.

```
name
↓
default
↓
upper
↓
화면 출력
```

20. URL Template Tag

HTML에서 URL을 직접 작성하는 것보다 Django의 URL 이름을 사용하는 것이 좋습니다. urls.py가 다음과 같다고 가정하겠습니다.

```
path("home/", views.home, name="home")
```

Template에서는:

```
<a href="{% url 'home' %}">홈</a>
```

사용할 수 있습니다.

직접 다음처럼 작성하는 것보다:

```
<a href="/home/">홈</a>
```

다음 방식이 권장됩니다.

```
<a href="{% url 'home' %}">홈</a>
```

URL 구조가 변경되어도 Template을 수정해야 할 가능성이 줄어들기 때문입니다.

21. URL에 매개변수 전달

게시글 상세 페이지가 다음과 같다면:

```
path(
    "board/<int:pk>/",
    views.detail,
    name="board_detail"
)
```

Template에서는 다음과 같이 사용할 수 있습니다.

```
<a href="{% url 'board_detail' board.id %}">
    {{ board.title }}
</a>
```

예를 들어 board.id가 10이라면 다음 URL이 만들어집니다.

```
/board/10/
```

22. app_name과 URL Namespace

실제 프로젝트에서는 app_name을 사용하는 것이 좋습니다.

board/urls.py

```
app_name = "board"

urlpatterns = [
    path("", views.list, name="list"),
    path("<int:pk>/", views.detail, name="detail"),
]
```

Template에서는:

```
<a href="{% url 'board:list' %}">
    게시판
</a>
```

상세 페이지는:

```
<a href="{% url 'board:detail' board.id %}">
    {{ board.title }}
</a>
```

와 같이 사용합니다.

23. CSRF Token

Django에서 POST 방식으로 Form을 전송할 경우 매우 중요합니다.

```
<form method="post">
    {% csrf_token %}

    <input type="text" name="title">
    <button type="submit">
        등록
    </button>
</form>
```

{% csrf_token %}은 **CSRF(Cross-Site Request Forgery)** 공격을 방지하기 위한 보안 토큰을 생성합니다.

Django Form에서 POST 요청을 사용하는 경우 기본적으로 반드시 넣는다고 이해하면 좋습니다.

24. Template 상속

DTL의 가장 중요한 기능 중 하나가 **Template 상속(Template Inheritance)**입니다.

웹사이트를 만들면 모든 페이지에서 다음과 같은 영역이 반복됩니다.

```
Header
Navigation
Content
Footer
```

모든 HTML 파일에 동일한 코드를 작성하면 유지보수가 어렵습니다.

따라서 공통 HTML을 base.html에 작성합니다.

```
templates/
|
|—— base.html
|
|—— home.html
|
|—— board/
|       |—— list.html
|       |—— detail.html
```

25. base.html 작성

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>
    {% block title %}
    Django Project
    {% endblock %}
  </title>
</head>

<body>
```

```
<header>
    <h1>Django Web Site</h1>
</header>

<nav>
    <a href="{% url 'home' %}">
        HOME
    </a>
</nav>

<main>
    {% block content %}

    {% endblock %}
</main>

<footer>
    Copyright 2026
</footer>
</body>
</html>
```

여기서 핵심은 다음 부분입니다.

```
{% block content %}

{% endblock %}
```

자식 Template이 내용을 삽입할 영역을 의미합니다.

26. {% extends %}

home.html에서 base.html을 상속할 수 있습니다.

```
{% extends "base.html" %}

{% block title %}
```

```
HOME
{% endblock %}

{% block content %}
<h2>홈 페이지</h2>
<p>환영합니다.</p>
{% endblock %}
```

그러면 실제 구조는 다음과 같이 만들어집니다.

```
base.html
|
|—— Header
|
|—— Navigation
|
|—— home.html의 content
|
|—— Footer
```

따라서 페이지마다 공통 레이아웃을 다시 작성할 필요가 없습니다.

27. block.super

부모 Template의 block 내용을 유지하면서 자식 내용을 추가하고 싶다면 block.super를 사용합니다.

부모:

```
{% block content %}
<p>기본 내용</p>
{% endblock %}
```

자식:

```
{% block content %}
{{ block.super }}
<p>추가 내용</p>
{% endblock %}
```

결과:

기본 내용

추가 내용

28. {% include %}

공통 HTML 조각을 별도의 Template으로 분리할 수도 있습니다.

예를 들어:

```
templates/  
|  
|—— base.html  
|  
|—— includes/  
|      |—— header.html  
|      |—— menu.html  
|      |—— footer.html
```

base.html:

```
{% include "includes/header.html" %}  
  
{% include "includes/menu.html" %}  
  
{% block content %}  
{% endblock %}  
  
{% include "includes/footer.html" %}
```

extends와 include의 차이를 이해하는 것이 중요합니다.

기능	목적
extends	부모 Template 상속
block	자식이 변경할 영역 지정
include	다른 HTML 조각 삽입

29. 로그인 사용자 처리

Django 인증 기능과 DTL을 결합하면 로그인 여부에 따라 화면을 다르게 구성할 수 있습니다.

```
{% if user.is_authenticated %}
    <p>
        {{ user.username }}님 환영합니다.
    </p>
{% else %}
    <p>
        로그인이 필요합니다.
    </p>
{% endif %}
```

여기에서:

```
user
```

는 현재 사용자 객체입니다.

```
user.is_authenticated
```

는 인증 여부를 나타냅니다.

30. 로그인/로그아웃 화면 구성 예

실제 프로젝트에서는 다음과 같이 활용할 수 있습니다.

```
{% if user.is_authenticated %}
    <span>
        {{ user.username }}님
    </span>

    <form
        method="post"
        action="{% url 'logout' %}">
        {% csrf_token %}

        <button type="submit">
```



```
        로그아웃
    </button>
</form>
{% else %}
    <a href="{% url 'login' %}">
        로그인
    </a>
{% endif %}
```

로그인한 경우:

```
홍길동님    [로그아웃]
```

로그아웃 상태라면:

```
[로그인]
```

처럼 구성할 수 있습니다.

31. Static 파일 사용

CSS, JavaScript, 이미지 등을 Template에서 사용할 때는 static을 이용합니다.

먼저 다음을 선언합니다.

```
{% load static %}
```

CSS:

```
<link
    rel="stylesheet"
    href="{% static 'css/style.css' %}">
```

JavaScript:

```
<script
    src="{% static 'js/main.js' %}">
</script>
```

이미지:

```

```

32. Template 주석

한 줄 주석은 다음과 같습니다.

```
{# 사용자 이름 출력 #}  
  
{{ user.username }}
```

여러 줄 주석은 다음과 같이 작성할 수 있습니다.

```
{% comment %}  
  
이 부분은 Template 주석입니다.  
브라우저 HTML 결과에도 출력되지 않습니다.  
  
{% endcomment %}
```

HTML 주석과 차이가 있습니다.

```
<!-- HTML 주석 -->
```

HTML 주석은 생성된 HTML 소스에 남을 수 있지만 Django Template 주석은 Template 처리 과정에서 제거됩니다.

33. with Tag

복잡하거나 긴 값을 임시 변수로 저장할 수 있습니다.

```
{% with total=boards|length %}  
  <p>  
    전체 게시글 : {{ total }}개  
  </p>  
{% endwith %}
```

34. Template에서 Django Form 출력

View에서 Form 객체를 전달하면:

```
form = MemberForm()

return render(
    request,
    "member/form.html",
    {"form": form}
)
```

Template에서는:

```
<form method="post">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">
        저장
    </button>
</form>
```

사용할 수 있습니다.

다음과 같은 방법도 있습니다.

```
{{ form.as_p }}
{{ form.as_table }}
{{ form.as_ul }}
```

35. 자동 HTML Escape

Django Template은 보안을 위해 HTML 문자를 기본적으로 Escape 처리합니다.

예를 들어 View에서:

```
message = "<script>alert('test')</script>"
```

를 전달하고:

```
{{ message }}
```

로 출력해도 일반적으로 <script>가 그대로 실행되지 않습니다.

이는 XSS(Cross-Site Scripting) 공격을 방지하는 데 중요한 역할을 합니다.

특히 신뢰할 수 있는 HTML을 그대로 출력해야 하는 경우 safe 필터가 있지만:

```
{{ content|safe }}
```

사용자 입력 데이터에 무분별하게 사용해서는 안 됩니다.

36. Custom Template Filter

Django에서 제공하는 Filter 이외에 개발자가 직접 Filter를 만들 수도 있습니다.

예를 들어 프로젝트 구조를 다음과 같이 구성할 수 있습니다.

```
board/  
|  
|—— templatetags/  
|   |—— __init__.py  
|   |—— board_filters.py
```

board_filters.py:

```
from django import template  
  
register = template.Library()  
  
@register.filter  
def won(value):
```

```
return f"{value:}원"
Template:
{% load board_filters %}

{{ price|won }}
```

price가 100000이라면:

```
100,000원
```

으로 출력할 수 있습니다.

37. Custom Template Tag

복잡한 출력 기능이 필요한 경우 개발자가 Template Tag를 직접 만들 수도 있습니다.

대표적으로 다음과 같은 종류가 있습니다.

```
Simple Tag
Inclusion Tag
Custom Filter
```

예를 들어:

```
@register.simple_tag
def site_name():
    return "Django Study"
Template:
{% load board_tags %}

{% site_name %}
```

38. View → Template 데이터 전달 과정

DTL을 이해할 때 가장 중요한 부분입니다.

View:

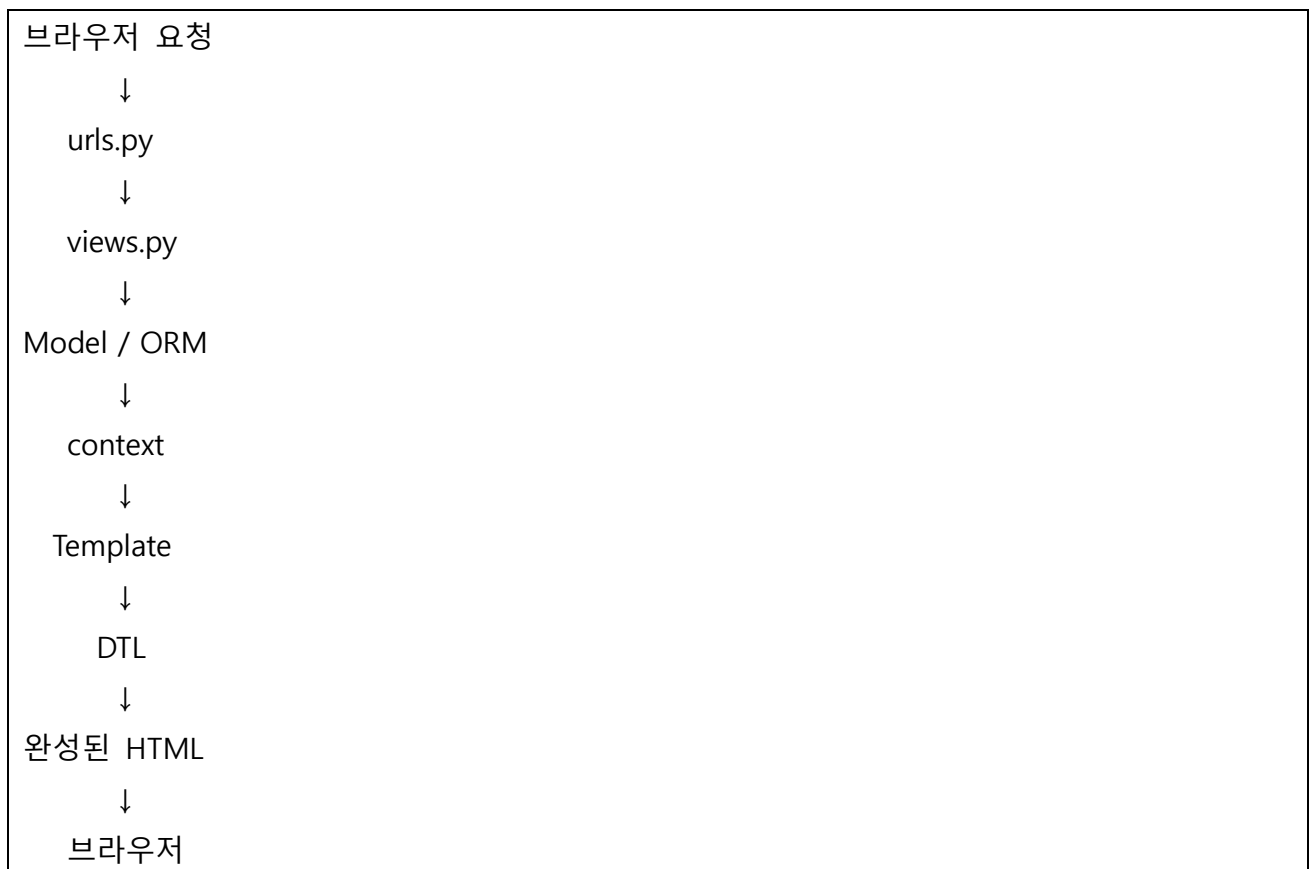
```
def home(request):
    context = {
        "name": "홍길동",
        "age": 25,
        "subjects": [
            "Python",
            "Django",
            "MySQL"
        ]
    }

    return render(
        request,
        "home.html",
        context
    )
```

Template:

```
<h1>
    {{ name }}님
</h1>
<p>
    나이 : {{ age }}
</p>
<ul>
{% for subject in subjects %}
    <li>
        {{ subject }}
    </li>
{% endfor %}
</ul>
```

전체 데이터 흐름은 다음과 같습니다.



39. 실제 게시판에서의 DTL 활용 예

게시판 목록 화면을 예로 들면 DTL의 핵심 기능이 거의 모두 사용됩니다.

```
{% extends "base.html" %}

{% block content %}
<h2>게시판</h2>

<table>
  <thead>
    <tr>
      <th>번호</th>
      <th>제목</th>
      <th>작성자</th>
      <th>등록일</th>
    </tr>
```

```

</thead>
<tbody>
{% for board in boards %}
    <tr>
        <td>
            {{ board.id }}
        </td>
        <td>
            <a href="{% url 'board:detail' board.id %}">
                {{ board.title }}
            </a>
        </td>
        <td>
            {{ board.writer.username }}
        </td>
        <td>
            {{ board.created_at|date:"Y-m-d" }}
        </td>
    </tr>

{% empty %}
    <tr>
        <td colspan="4">
            등록된 게시글이 없습니다.
        </td>
    </tr>
{% endfor %}
</tbody>
</table>
{% endblock %}

```

이 짧은 코드 안에서도 다음 DTL 기능들이 사용됩니다.

```

{% extends %}
↓
Template 상속

```


{% block %}



화면 영역 정의

{% for %}



게시글 반복

{{ board.id }}



변수 출력

{% url %}



URL 생성

|date



Filter

{% empty %}



데이터가 없는 경우 처리

40. DTL에서 하지 않는 것이 좋은 작업

DTL은 Python 프로그램을 작성하는 공간이 아닙니다.

따라서 복잡한 작업은 Template에서 처리하지 않는 것이 좋습니다.

다음과 같은 로직은 View, Model, Service 계층 등에서 처리하는 것이 적절합니다.

- 복잡한 데이터 계산
- Database 직접 접근
- 복잡한 QuerySet 처리
- 업무 규칙 처리
- 복잡한 문자열 처리
- 외부 API 호출

권장 구조는:



입니다.

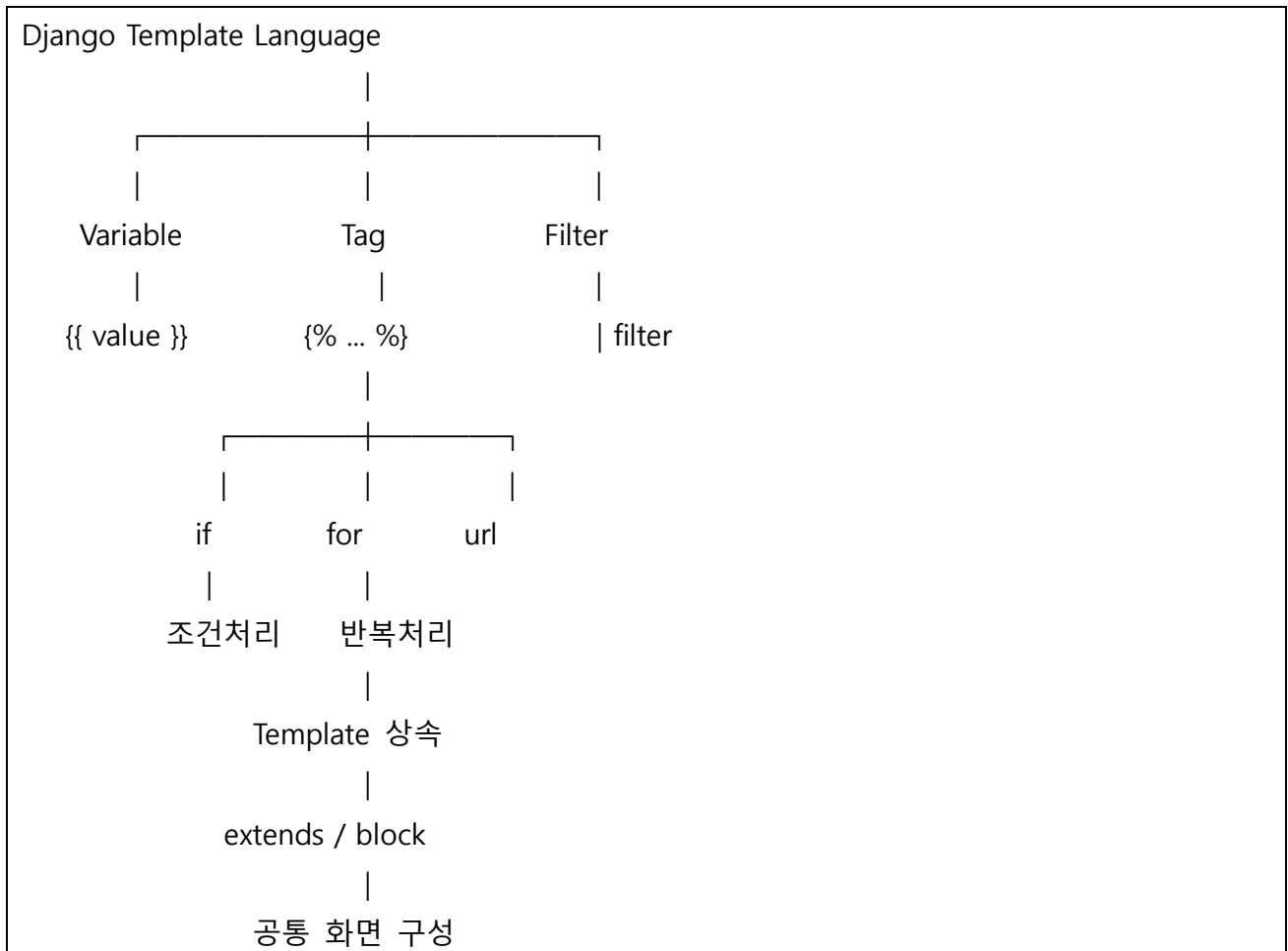
41. Django Template의 핵심 문법 정리

기능	DTL 문법
변수 출력	{{ variable }}
객체 속성	{{ user.username }}
조건문	{% if %}
조건 종료	{% endif %}
반복문	{% for %}
반복 종료	{% endfor %}
빈 목록 처리	{% empty %}
URL	{% url %}
CSRF	{% csrf_token %}
Static	{% load static %}
Template 상속	{% extends %}
영역 정의	{% block %}
다른 Template 포함	{% include %}
Filter	{{ value filter }}
한 줄 주석	{# #}

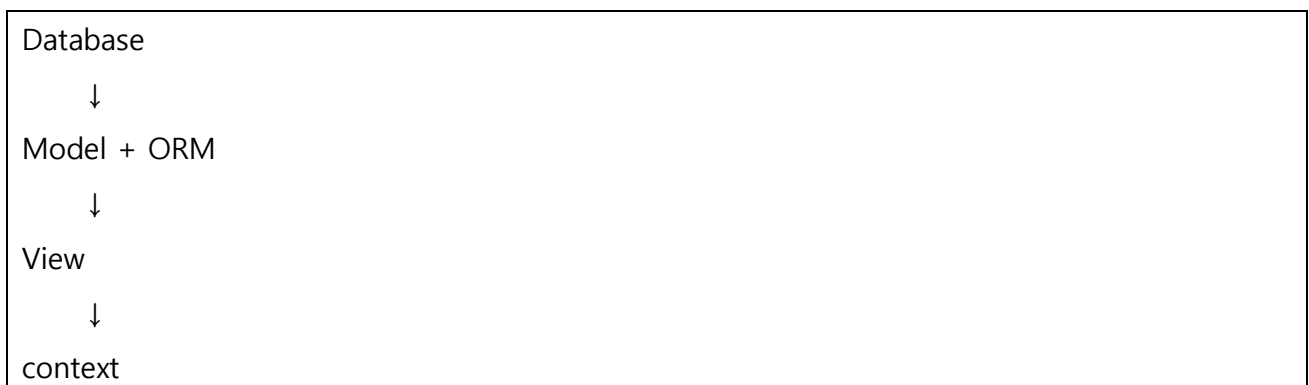
기능	DTL 문법
여러 줄 주석	{% comment %}

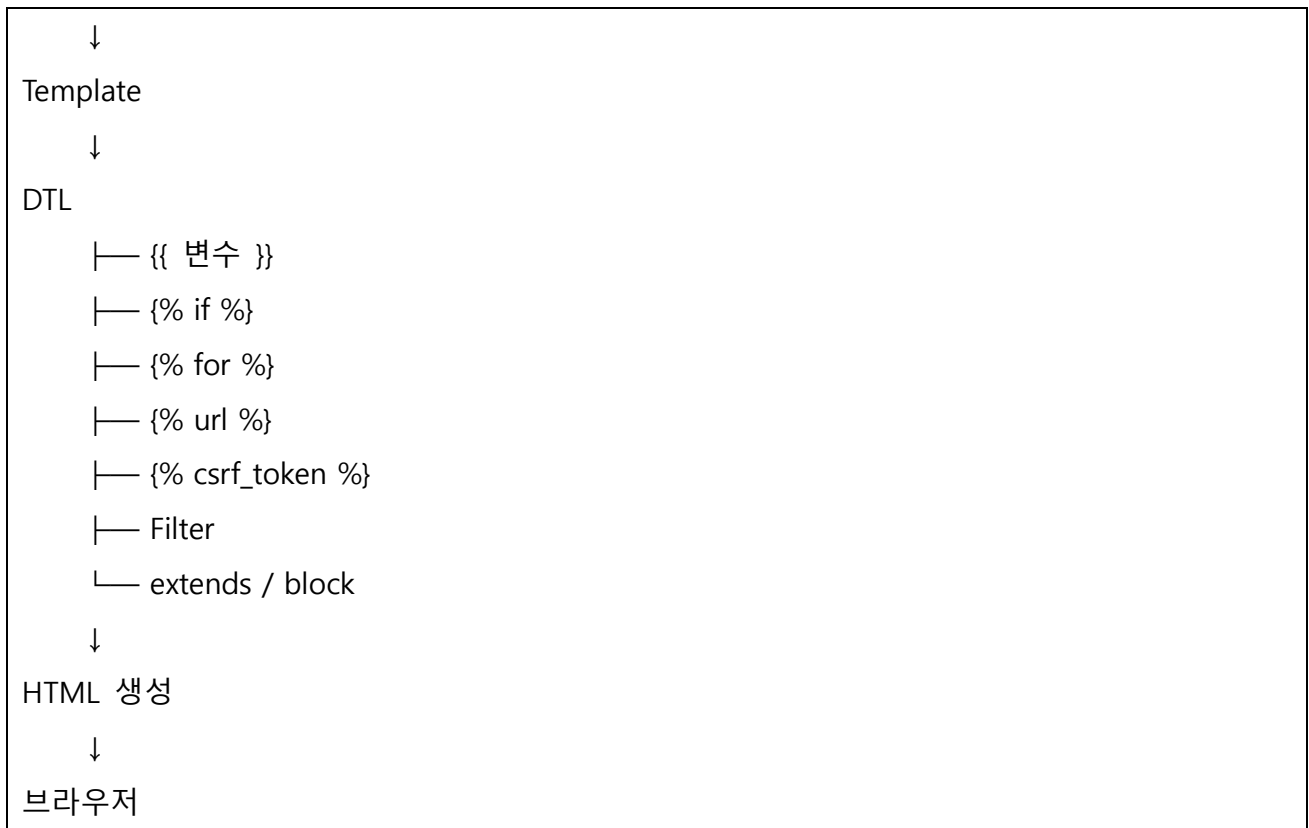
42. DTL의 핵심 개념

Django Template Language를 처음 학습할 때는 다음 흐름으로 이해하는 것이 가장 중요합니다.



실제 Django 프로젝트에서는 결국 다음과 같은 형태로 연결됩니다.





따라서 DTL 학습에서는 ① 변수 출력 → ② 조건문 → ③ 반복문 → ④ Filter → ⑤ URL → ⑥ Form/CSRF → ⑦ Template 상속 → ⑧ 로그인 사용자 처리 → ⑨ 게시판 CRUD 화면 적용 순서로 익히면 Django의 프론트 화면 구성을 체계적으로 이해할 수 있습니다.

4. Model 과 ORM

1. Model

Model은 데이터베이스 테이블을 Python 클래스로 표현하는 기능입니다.

예를 들어 다음과 같은 게시판 테이블이 있다고 가정하겠습니다.

```
BOARD

id
title
content
writer
created_at
```

Django에서는 다음처럼 작성할 수 있습니다.

```
from django.db import models

class Board(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    writer = models.CharField(max_length=50)
    created_at = models.DateTimeField(
        auto_now_add=True
    )
```

Django가 이 클래스를 데이터베이스 테이블로 변환해 줍니다.

즉,

```
Python Class
  ↓
Database Table
```

관계입니다.

2. Migration

Model을 만들었다고 데이터베이스 테이블이 바로 생성되는 것은 아닙니다.

Django에서는 **Migration** 과정을 수행합니다.

첫 번째 명령:

```
python manage.py makemigrations
```

Django가 Model 변경사항을 확인하여 Migration 파일을 생성합니다.

그 다음:

```
python manage.py migrate
```

실제 데이터베이스에 테이블을 생성합니다.

과정을 정리하면 다음과 같습니다.

```
models.py 작성
    ↓
makemigrations
    ↓
Migration 파일 생성
    ↓
migrate
    ↓
Database Table 생성
```

이 과정은 Django에서 매우 중요합니다.

3. Django ORM

Django의 가장 중요한 기능 중 하나가 **ORM**입니다.

ORM은 **Object Relational Mapping**의 약자입니다.

쉽게 말하면 SQL을 직접 작성하지 않고 Python 코드로 데이터베이스를 사용할 수 있는 기능입니다.

예를 들어 SQL에서 다음과 같이 조회한다고 가정합니다.

```
SELECT *  
FROM board;
```

Django ORM에서는:

```
Board.objects.all()
```

로 사용할 수 있습니다.

데이터 한 건 조회

```
SQL:  
SELECT *  
FROM board  
WHERE id = 1;
```

Django:

```
Board.objects.get(id=1)
```

또는:

```
Board.objects.get(pk=1)
```

데이터 추가

```
Board.objects.create(  
    title="Django 공부",  
    content="Django ORM 공부",  
    writer="홍길동"  
)
```

데이터 수정

```
board = Board.objects.get(pk=1)
board.title = "수정된 제목"
board.save()
```

데이터 삭제

```
board = Board.objects.get(pk=1)
board.delete()
```

따라서 Django에서는 SQL을 잘 모르더라도 기본적인 CRUD를 구현할 수 있습니다.

4. CRUD

웹 개발에서 가장 중요한 개념 중 하나가 CRUD입니다.

CRUD는 다음 네 가지 기능을 의미합니다.

CRUD	의미	SQL
Create	등록	INSERT
Read	조회	SELECT
Update	수정	UPDATE
Delete	삭제	DELETE

게시판으로 생각하면:

```
Create → 게시글 작성
Read → 게시글 목록 / 상세보기
Update → 게시글 수정
Delete → 게시글 삭제
```

입니다.

Django ORM을 사용하면 다음과 같이 처리합니다.

```
# Create
Board.objects.create(...)
```



```
# Read
Board.objects.all()

# Update
board.save()

# Delete
board.delete()
```

5. Form 처리

1. Form 처리

웹사이트에서는 사용자로부터 많은 데이터를 입력받습니다.

예:

회원가입
로그인
게시글 작성
댓글 작성
상품 등록

Django는 이를 위해 Form 기능을 제공합니다.

예:

```
from django import forms

class BoardForm(forms.Form):
    title = forms.CharField()
    content = forms.CharField(
        widget=forms.Textarea
    )
```

그러나 Model과 연결된 경우에는 ModelForm을 많이 사용합니다.

```
from django import forms
from .models import Board

class BoardForm(forms.ModelForm):
    class Meta:
        model = Board
        fields = [
            "title",
            "content"
        ]
```

이렇게 하면 Model을 기준으로 HTML 입력 폼을 자동으로 만들 수 있습니다.

2. GET과 POST 처리

웹에서는 데이터를 주고받을 때 GET과 POST를 많이 사용합니다.

GET

주로 데이터를 조회합니다.

예:

```
/board/  
/board/10/
```

POST

주로 데이터를 등록하거나 변경합니다.

예:

```
회원가입  
로그인  
게시글 작성  
게시글 수정
```

Django에서는 다음과 같이 구분합니다.

```
def board_create(request):  
  
    if request.method == "POST":  
        print("POST 요청")  
    else:  
        print("GET 요청")
```

게시글 작성 화면을 처음 열면:

```
GET
```

작성 버튼을 누르면:

```
POST
```

가 전달되는 구조입니다.

3. request 객체

View 함수의 다음 부분을 많이 보게 됩니다.

```
def board_list(request):
```

여기서 request는 사용자가 서버로 전달한 요청 정보입니다.
request에는 다양한 정보가 들어 있습니다.

예:

```
request.method  
현재 요청 방식:  
GET  
POST  
  
확인 가능.  
POST 데이터:  
request.POST  
  
GET 데이터:  
request.GET  
  
로그인 사용자:  
request.user  
  
세션:  
request.session
```

따라서 Django View에서 매우 중요한 객체입니다.

4. redirect

작업이 완료된 후 다른 페이지로 이동시킬 때 사용합니다.

예:

```
from django.shortcuts import redirect  
  
def board_create(request):
```

...

```
return redirect("board:list")
```

예를 들어 게시물 등록 후:

게시글 작성

↓

DB 저장

↓

게시글 목록으로 이동

형태로 사용할 수 있습니다.

6. Django Admin 과 권한 및 Session

1. Django Admin

Django의 매우 강력한 기능 중 하나가 **관리자 페이지**입니다.

직접 관리자 페이지를 개발하지 않아도 기본 관리 시스템을 제공합니다.

관리자 계정을 생성합니다.

```
python manage.py createsuperuser
```

서버 실행:

```
python manage.py runserver
```

브라우저:

```
http://127.0.0.1:8000/admin/
```

접속하면 관리자 로그인 화면이 나타납니다.

Model을 관리자 페이지에서 관리하려면:

```
from django.contrib import admin
from .models import Board

admin.site.register(Board)
```

만 등록하면 됩니다.

관리자는 브라우저에서 직접:

```
조회
등록
수정
삭제
```

할 수 있습니다.

2. Django 인증 시스템

Django는 로그인 기능도 기본 제공합니다.

대표적으로 다음 기능이 있습니다.

- 회원
- 비밀번호
- 로그인
- 로그아웃
- 세션
- 권한
- 그룹
- 관리자

Django 기본 User 모델을 사용할 수 있습니다.

```
from django.contrib.auth.models import User
```

회원 생성:

```
User.objects.create_user(  
    username="hong",  
    password="1234"  
)
```

비밀번호는 일반 문자열로 저장되지 않고 암호화되어 저장됩니다.

3. 로그인

Django에서는 `authenticate()`를 통해 사용자 정보를 확인합니다.

```
from django.contrib.auth import authenticate  
  
user = authenticate(  
    request,  
    username="hong",  
    password="1234"  
)
```

사용자가 존재하고 비밀번호가 맞으면:

```
user
```

객체가 반환됩니다.

로그인을 실제 세션에 등록하려면:

```
from django.contrib.auth import login  
  
login(request, user)
```

사용자는 이제 로그인 상태가 됩니다.

4. 로그아웃

로그아웃은 매우 간단합니다.

```
from django.contrib.auth import logout  
  
logout(request)
```

로그인 세션이 제거됩니다.

5. 로그인 여부 확인

Django에서는 현재 사용자가 로그인 상태인지 쉽게 확인할 수 있습니다.

Python:

```
request.user.is_authenticated
```

Template:

```
{% if user.is_authenticated %}  
    {{ user.username }}님 환영합니다.  
{% else %}  
    로그인하세요.  
{% endif %}
```


6. 로그인 사용자만 접근하도록 제한

게시글 등록 페이지처럼 로그인 사용자만 접근하도록 만들 수도 있습니다.

```
from django.contrib.auth.decorators import login_required

@login_required
def board_create(request):

    ...
```

로그인하지 않은 사용자가 해당 페이지에 접속하면 로그인 페이지로 이동합니다.

클래스 기반 View에서는:

```
LoginRequiredMixin
```

을 사용할 수도 있습니다.

7. 권한 관리

Django는 사용자별 권한 기능도 제공합니다.

예:

```
게시글 조회 권한
게시글 등록 권한
게시글 수정 권한
게시글 삭제 권한
```

관리자인지 확인하려면:

```
request.user.is_staff
```

최고 관리자라면:

```
request.user.is_superuser
```

를 사용할 수 있습니다.

예를 들어 관리자에게 모든 게시물 삭제 권한을 주고 싶다면 다음과 같은 형태로 구현할 수 있습니다.

```
if (
    board.writer == request.user
    or request.user.is_staff
):

    board.delete()
```

즉: 본인 작성자 OR 관리자이면 삭제가 가능합니다.

8. Session

Django는 서버에서 사용자 상태를 유지하기 위한 Session 기능을 제공합니다.

예를 들어:

```
request.session["username"] = "hong"
```

세션 값 읽기:

```
username = request.session.get("username")
```

삭제:

```
del request.session["username"]
```

다만 Django 기본 로그인 시스템을 사용하면 대부분의 경우 세션을 직접 처리하지 않아도 됩니다.

```
login(request, user)
```

를 호출하면 Django가 인증 세션을 관리합니다.

7. 파일 관계 및 보안

1. Static File

웹사이트에서는 다음 파일을 사용합니다.

- CSS
- JavaScript
- 이미지

이러한 파일을 Django에서는 **Static File**이라고 합니다.

예:

```
static/  
├── css/  
│   └── style.css  
├── js/  
│   └── script.js  
└── images/  
    └── logo.png
```

HTML에서는:

```
{% load static %}  
  
<link  
    rel="stylesheet"  
    href="{% static 'css/style.css' %}"  
>
```

형태로 사용합니다.

2. Media File

사용자가 업로드하는 파일은 Static File과 구분합니다.

예:

- 프로필 사진
- 게시판 첨부파일
- 상품 이미지

이러한 파일을 **Media File**이라고 합니다.

Model 예:

```
class Member(models.Model):  
  
    photo = models.ImageField(  
        upload_to="member/"  
    )
```

파일이 업로드되면 예를 들어 다음 위치에 저장할 수 있습니다.

```
media/member/photo.jpg
```

3. Middleware

Middleware는 사용자의 요청과 응답 사이에서 공통 작업을 처리합니다.

구조는 다음과 같습니다.

```
Browser  
  ↓  
Middleware  
  ↓  
View  
  ↓  
Middleware  
  ↓  
Browser
```

Django에서는 기본적으로 다음과 같은 기능이 Middleware를 통해 처리됩니다.

- Session
- Authentication
- CSRF
- Security
- Message

settings.py에서 확인할 수 있습니다.

```
MIDDLEWARE = [  
    ...  
]
```

4. CSRF 보안

Django는 기본적으로 CSRF 공격을 방지합니다.

HTML Form에서 POST 요청을 보낼 때 다음 코드를 사용합니다.

```
<form method="post">  
  
    {% csrf_token %}  
  
    ...  
  
</form>
```

이 코드를 빠뜨리면 보통 다음과 같은 오류가 발생할 수 있습니다.

```
403 Forbidden  
CSRF verification failed
```

따라서 Django Form에서 POST를 사용할 때:

```
{% csrf_token %}
```

은 매우 중요합니다.

5. Database 연동

Django는 기본적으로 SQLite를 사용합니다.

settings.py

```
DATABASES = {  
    "default": {  
        "ENGINE": "django.db.backends.sqlite3",  
        "NAME": BASE_DIR / "db.sqlite3",  
    }  
}
```

하지만 MySQL, PostgreSQL 등도 사용할 수 있습니다.

예를 들어 MySQL:

```
DATABASES = {  
    "default": {  
        "ENGINE":  
            "django.db.backends.mysql",  
  
        "NAME":  
            "django_db",  
  
        "USER":  
            "django",  
  
        "PASSWORD":  
            "1234",  
  
        "HOST":  
            "localhost",  
  
        "PORT":  
            "3306",  
    }  
}
```

그 후 Django ORM은 동일한 방식으로 사용할 수 있습니다.

```
Board.objects.all()
```

즉, Django ORM을 사용하면 데이터베이스가 바뀌어도 Python 코드의 상당 부분을 그대로 사용할 수 있습니다.

6. Django Shell

Django에서는 Python Shell을 이용하여 Model과 ORM을 직접 테스트할 수 있습니다.

실행:

```
python manage.py shell
```

Model 가져오기:

```
from board.models import Board
```

게시글 조회:

```
Board.objects.all()
```

게시글 등록:

```
Board.objects.create(  
    title="첫 번째 글",  
    content="Django 공부"  
)
```

ORM 학습에 매우 유용합니다.

7. 함수 기반 View와 클래스 기반 View

Django View는 크게 두 가지 방식으로 작성할 수 있습니다.

함수 기반 View

Function Based View, FBV

```
def board_list(request):  
  
    boards = Board.objects.all()  
  
    return render(  
        request,  
        "board/list.html",  
        {  
            "boards": boards  
        }  
    )
```

구조가 명확해서 처음 Django를 배울 때 이해하기 좋습니다.

클래스 기반 View

Class Based View, CBV

```
from django.views.generic import ListView  
  
class BoardListView(ListView):  
  
    model = Board
```

공통 코드를 줄일 수 있어 실무에서 많이 사용합니다.

처음 학습할 때는 **FBV** → **CBV** 순서로 배우는 것이 이해하기 쉽습니다.

8. Django의 핵심 파일 관계

게시판 App을 만든다고 하면 다음 파일들이 핵심입니다.

```
board/
|
├── models.py
|
├── views.py
├── urls.py
├── forms.py
├── admin.py
|
└── templates/
    ├── board/
    │   ├── list.html
    │   ├── detail.html
    │   └── form.html
```

역할은 다음과 같습니다.

파일	역할
models.py	데이터베이스
views.py	요청 처리
urls.py	URL 연결
forms.py	입력 Form
admin.py	관리자 페이지
templates	HTML 화면

이 구조를 이해하면 Django 전체 구조의 상당 부분을 이해한 것입니다.

9. 게시판 요청이 처리되는 실제 과정

게시글 목록 조회를 예로 들겠습니다.

사용자가:

```
http://localhost:8000/board/
```

접속합니다.

① Browser 요청

```
GET /board/
```

② 프로젝트 urls.py

```
path(
    "board/",
    include("board.urls")
)
```

③ board/urls.py

```
path(
    "",
    views.board_list,
    name="list"
)
```

④ views.py

```
def board_list(request):

    boards = Board.objects.all()

    return render(
        request,
        "board/list.html",
        {
```

```
        "boards": boards
    }
)
```

⑤ models.py

```
Board.objects.all()
```

데이터베이스에서 게시글을 조회합니다.

⑥ Template

```
{% for board in boards %}

    <p>
        {{ board.title }}
    </p>

{% endfor %}
```

⑦ 브라우저 출력

```
Django 공부
Python 공부
MySQL 공부
```

따라서 Django 웹 개발의 핵심 흐름은 다음 한 줄로 정리할 수 있습니다.

URL → View → Model → Database → View → Template